EEG/FNIRS prototype.

We are trying to create a low cost, fully open source, multimodal brain sensing device that uses 3 or 4 photodiode channels and 4 active electrode channels to connect to our nRF53 board, which features an ADS131M08. We have tested and confirmed both the FNIRS and EEG quality independently, but we're trying to integrate this into a final design. We specifically need to have the LEDs flash in correspondence with the ADS131 sampling rate so that the photodiodes are cleanly sampling LED pulses, synced to the 250SPS-configured ADS131. This will allow us to sample as fast as the ADC allows. The LED pulses will be driven by a miniature $4 RP2040 board on the side which will be commanded by the main nRF53 board.
Based on: https://github.com/joshbrew/nRF5x-Biosensing-Boards

So main todo is:

1: Sync LED pulses on a timing block with the ADC readings, so LEDs are part-on then part-off during a sampling period for cleanly sampling single LED pulses. At 250sps this is 4ms per LED pulse period (see below for more details).

2: Test EEG electrodes. I didn't get an output. You should see blinks amplified at 100X from the base voltage, which is about 200uV. You can turn on a digital bandpass filter in the test software we linked.

3. Test PD readings and Electrode readings running simultaneously to see if we can manage both on a single ADC. The EEG amplifiers should make sure the signals are far stronger than system noise from the LEDs or ambient noise in the room.

Provided:

nRF53 Microcontroller and Dev Kit (programmed via nRF
PDs and LEDs
Electrode circuit PCBs and parts
Schematics, existing firmware

Needed back:
Modified firmware and diagrams
Any design corrections required

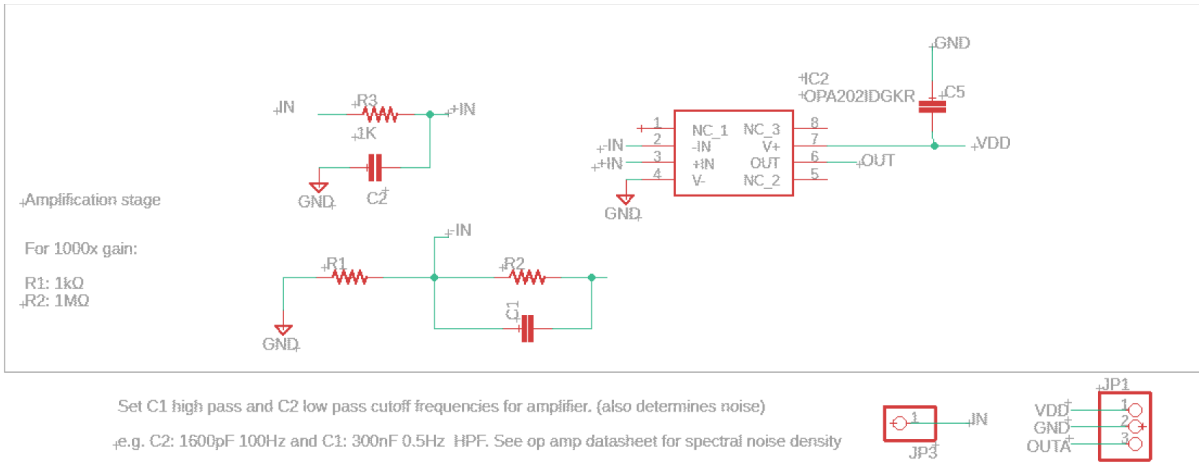
And then I may recruit you for the flex PCB drawing depending, but I can do that part myself, while I just need to finalize the schematics and firmware right now to be able to move along with testing. I am working right now on the RP2040 portion of the firmware and UART communication.

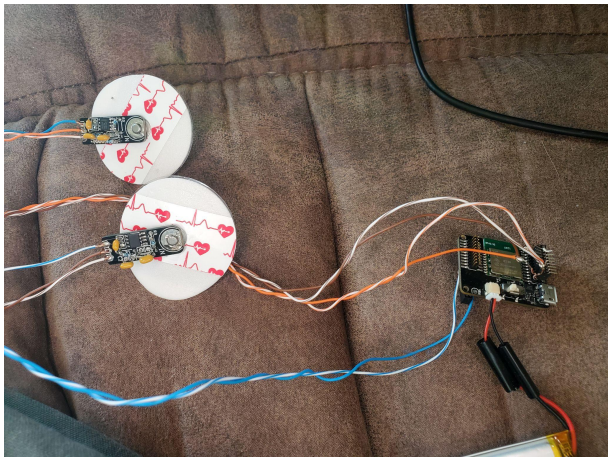
EEG sensors:
https://github.com/joshbrew/EEG-Amplifier/tree/main

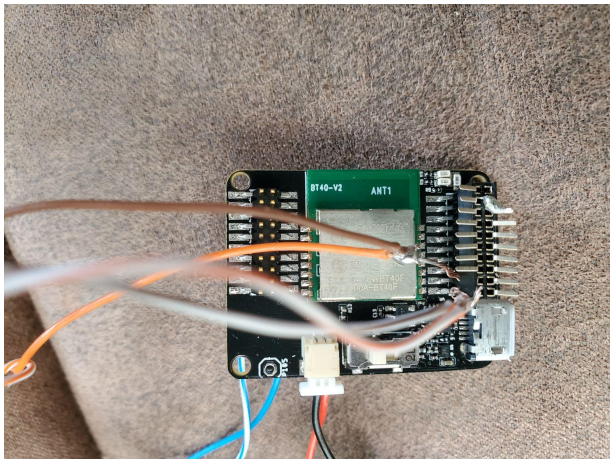
EEG signals are very weak, on the order of nanovolts and microvolts, so we are using low noise OPA202 amplifiers in a non-inverting config to amplify EEG bandpowers, as our ADC is only just good enough to do alpha and weak beta EEG signals, verified. The OPA202 also ensures that it will be less sensitive to system noise like from the power draw of the LEDs.

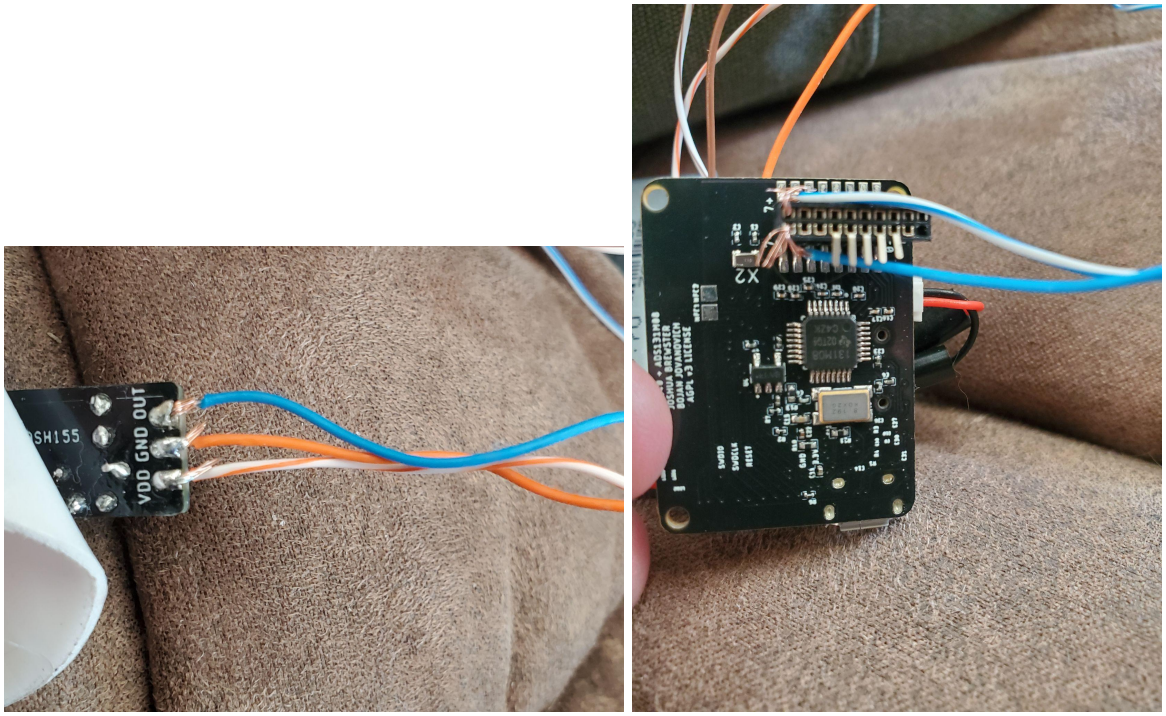
Here is how they are hooked up in our test setup, but we so far did not get any output, though it is probably our solder job:

Let's use 100X amplification as the ADS131 is already amplifying the signal 32X, EEG signals are nanovolts up to ~20 microvolts. The OPA202 is good for narrow band low freq amplification.









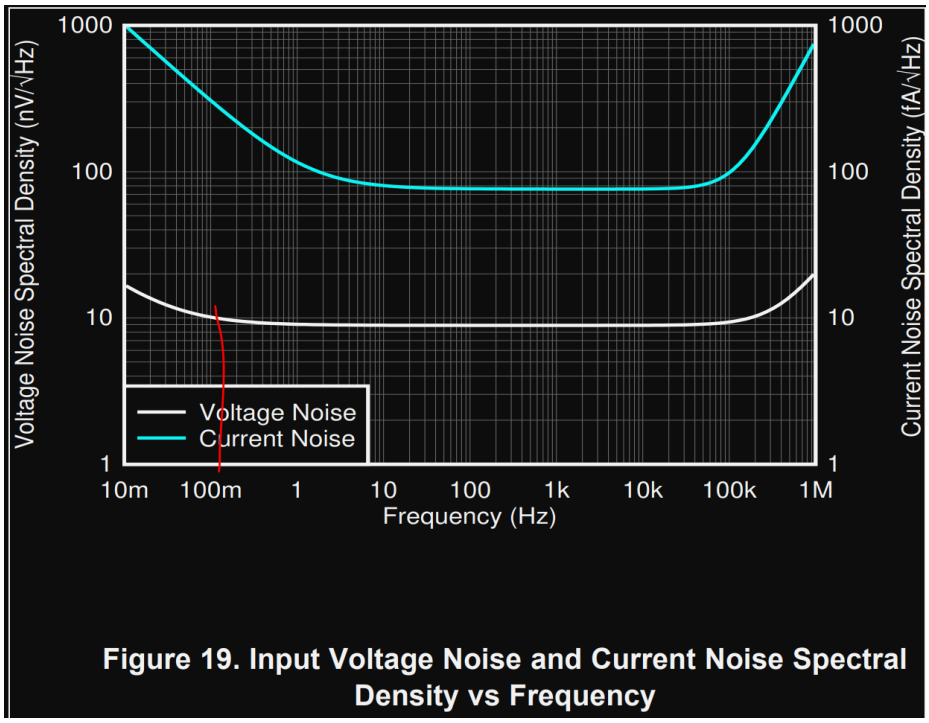
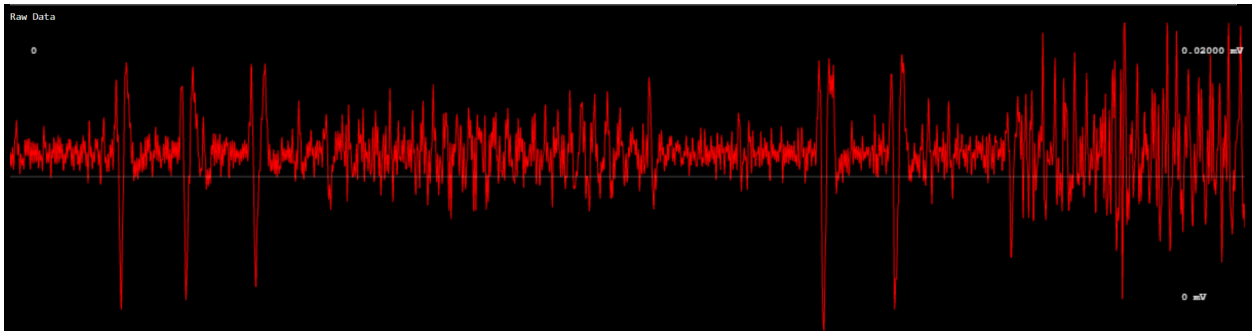


Figure 19. Input Voltage Noise and Current Noise Spectral Density vs Frequency

OPA202 noise density works out to about ~112nVRMS for ~0-100Hz. Our base ADS131M08 noise level is about 350nVRMS, but the input impedance is worse (1MΩ) so the signal is not as amplified. OPA202 input impedance is about 3TΩ, more than enough for EEG, and was designed for such tasks.

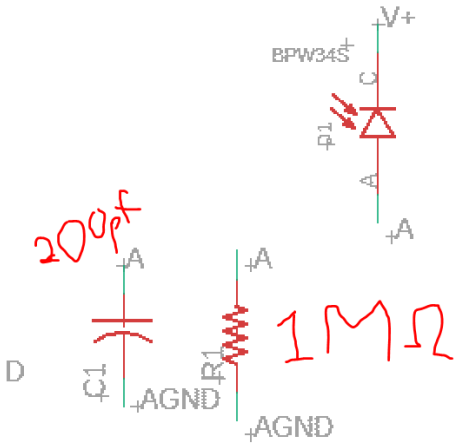
EEG output (alpha waves and blinks):

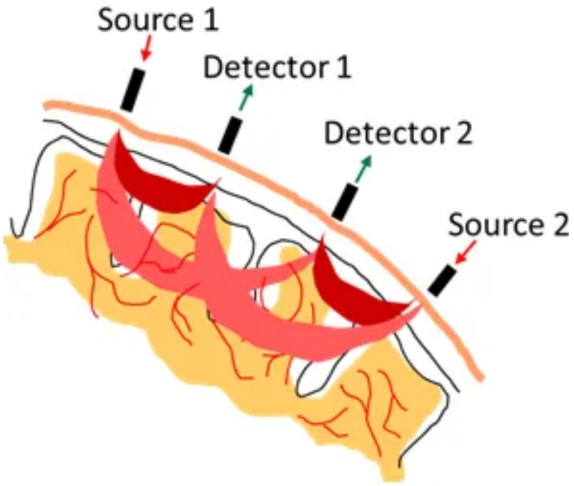


If you get this you will know it's working, I'll teach you how to generate Alpha with your brain, it's kind of stupid. I got this signal off the channels without extra amplification so you can get this right away, we need the op amps however to improve input impedance and make the weaker beta/gamma signals more parseable. You should see more or less the same output but amplified through the OPA202. We put it at 100x amplification in the

FNIRS sensors:

Simple photodiode circuit. "A" connects to the + input on an ADC channel, and the - input to GND.





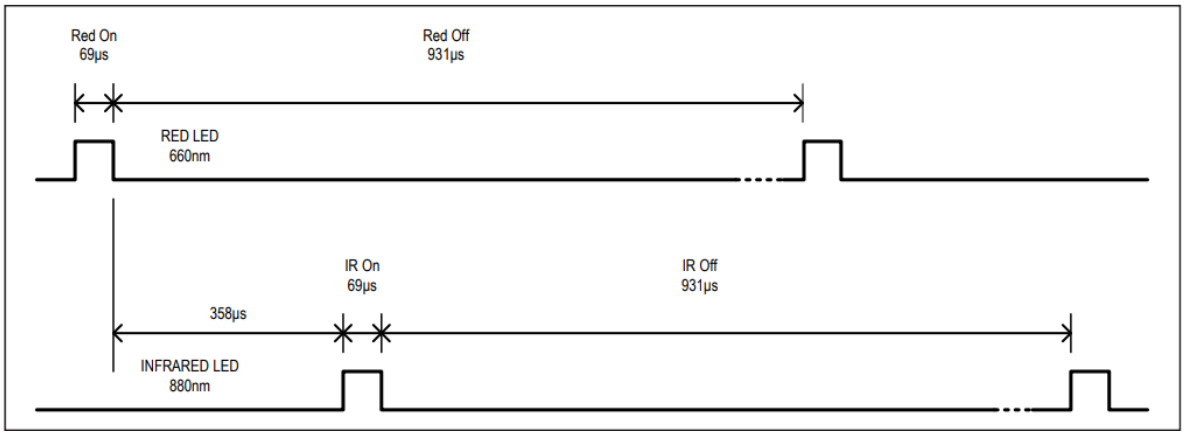
The LED is driven at 3.3V on the RP2040, just place a resistor on the desired GPIO to drive the LED at about 20mA. Higher wavelengths give better signal but it can irritate your eyes. The LEDs are **~740nm Far Red** and **~850nm Infrared** LEDs. Far Red has better penetration than Red for darker skin tones, as melanin reflects red light.These are standard wavelengths for pulse oximetry, which FNIRS is just a different version of. Google "Pulse Oximeter Fundamentals" to learn more. The LEDs are placed about 25-30mm from the PD on single

channel versions, but can be interwoven at varying distances to create a form of 3D tomography which is one version we are working on.

The LED pulses will occur like this (taken from the MAX30102 datasheet). We'll dedicate the 4 dollar Waveshare RP2040-Zero for this task as the nRF53 microcontroller cannot maintain sync while running other tasks. We looked at other LED drivers and this stood out as the best option. The RP2040 driver can be found in the nrf5340 driver section of the nrf5x biosensing board repo. If you name it main.py it'll become the default program on the RPi but careful as it might brick it. You can unbrick it by dragging and dropping this file to the USB drive that shows up for the device:
https://drive.google.com/file/d/1gL0tzlR-D-wr0kTru96ldV-nZX4Uoxm8/view?usp=sharing

Table 14. Slot Timing

PULSE-WIDTH SETTING (µs)	CHANNEL SLOT TIMING (TIMING PERIOD BETWEEN PULSES) (µs)	CHANNEL-CHANNEL TIMING (RISING EDGE-TO-RISING EDGE) (µs)
69	358	427
118	407	525
215	505	720
411	696	1107



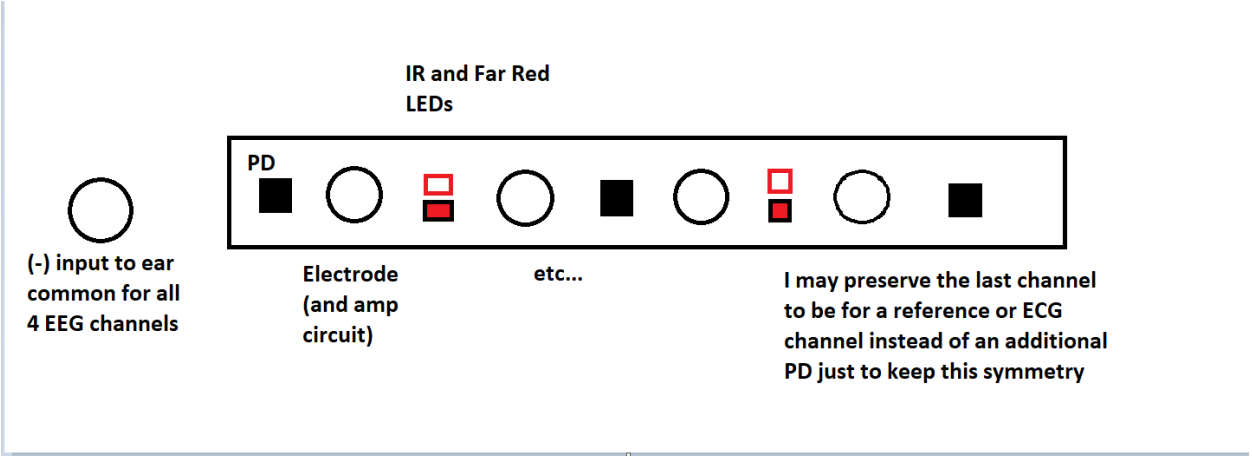
Now, based on our sampling rates on the ADC, the LED flash rates are:

```
## Sample rates
1000/250sps = 4ms per sample. (current target)
500sps = 2ms per sample
1000sps = 1ms per sample
2000sps = .5ms per sample
```

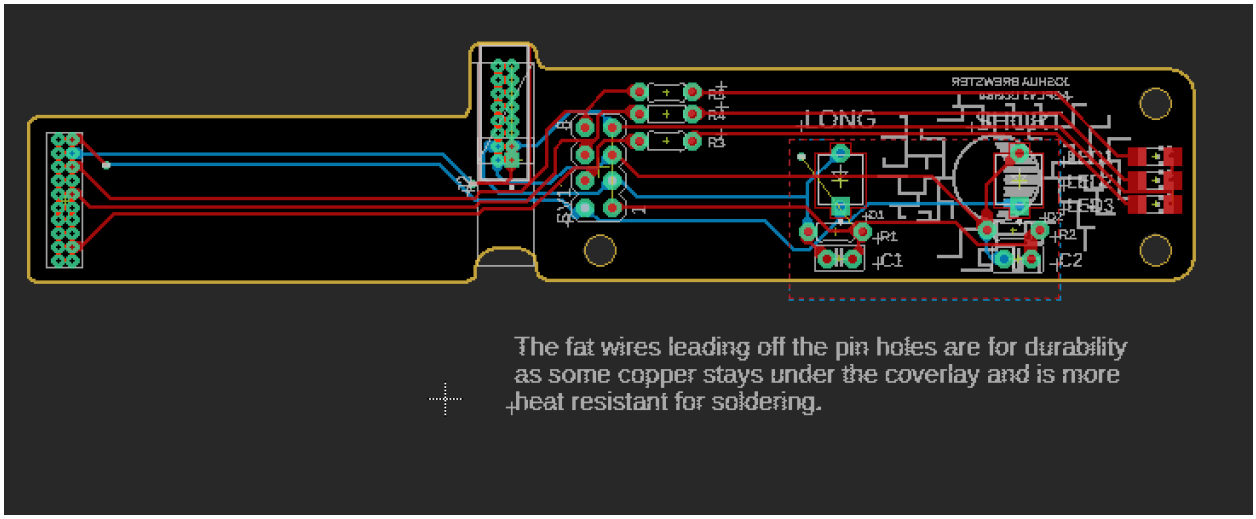
Each LED flash then needs to occur within a 4ms time window synced to the 250sps rate. This should be pretty easy, and otherwise it only needs to illuminate for a portion of that as LEDs have a rise and fall time, which is why power is only supplied for a portion of the sampling period. The RP2040 will just turn on, delay X amount, then turn off, and wait for the next loop to occur to switch the next LED on. Just some simple micropython, then the nRF5340 needs to

report which GPIO have been turned on, just using a preprogrammed buffer and knowing when the LEDs are turned on/off. We are working on this firmware so the commands are ready to send, but we need help with the sync and testing.

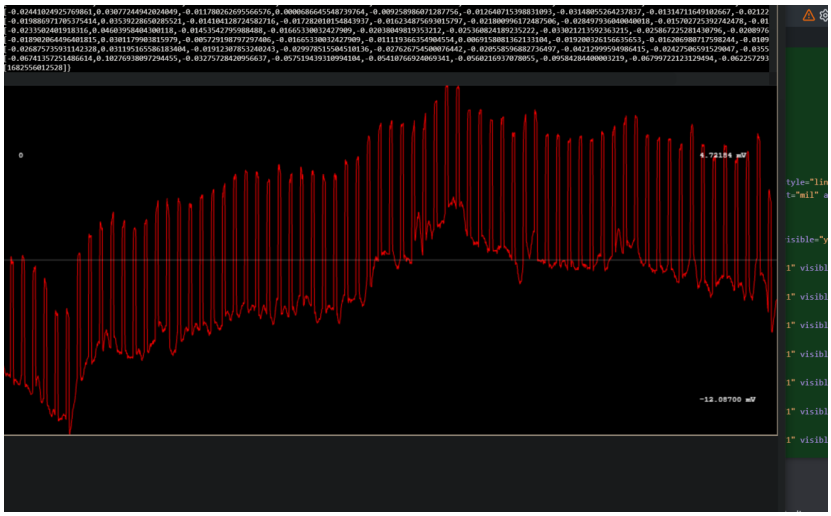
The final layout for our 4x4 board will be something like this:



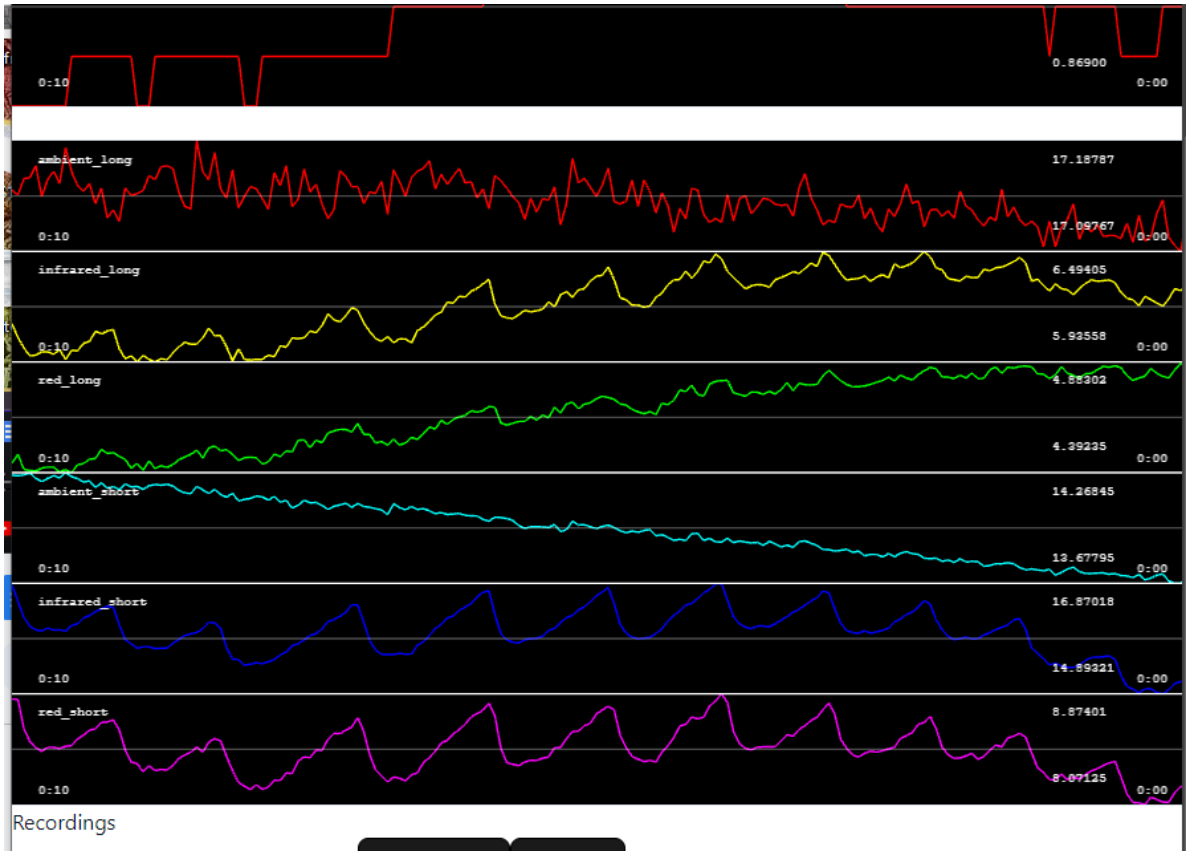
2 Channel FNIRS prototype (1 3cm channel and 1.5cm channel). This is what I sent. Use the RP2040 to hook to the LED power inputs as shown in this schematic. I hooked up some breakout pins to plug the RP2040 to the LEDs from here, then we just need to power the RP2040 from the nRF5340 and enable the LED routine via UART at the right time to sync LED pulses:



When alternating LEDs, the below image is the kind of signal we see raw before processing into each wavelength/ambient stream like above:



Sample data outpu on 2 channel prototypet after processing LED pulses into correct sequence (just in a web demo). Triangle waves in lines 2,3, 5, and 6 are heart rate PPG waves:

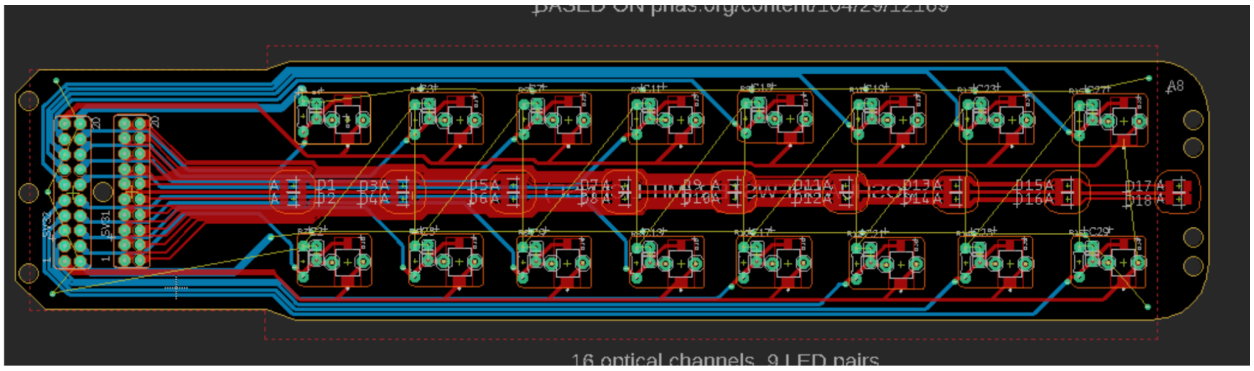


This was only running around ~20 LED pulses per second as the nRF5340 cannot drive the LEDs with precise pulse and still run all the BLE sensor services

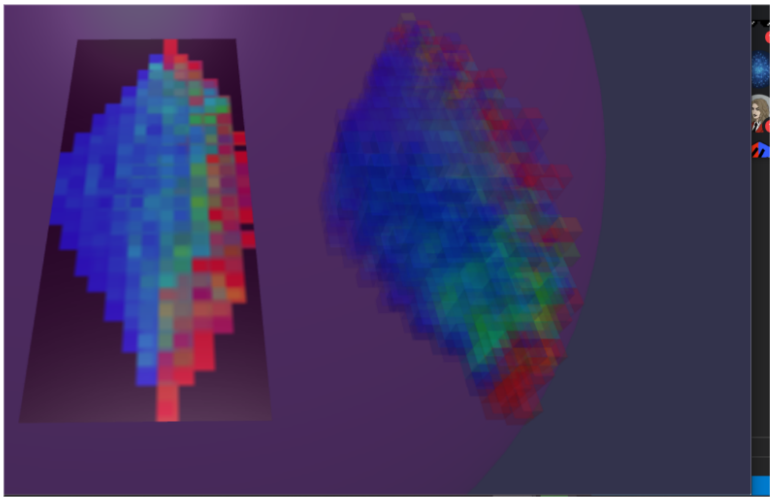
I'll share this source code for this chart for more testing, however for now you can use https://devicedebugger.netlify.app and just try to get a single LED pulse output to fit to a single ADC sample. The site is preconfigured for our BLE device's characteristics, then just subscribe to the first notification and set the decoder to ADS131M08. Our data is, even at this quality which in my opinion is quite low, still better than what a PhD from Vanderbilt labs was getting so if we just fix the LED problem and double down with getting EEG readings on the amps working simultaneously off the same ADC (if the LEDs don't interfere too much anyway, we don't know yet) then we are going to have state of the art brain sensors.

Fun stuff! This has all already been open sourced but we just need to actually fix the problems and then demonstrate it to peers, including a professor in Islamabad who is on the IEEE board for Pakistan and is an expert in this field.

16 channel prototype (not sent). If we fix the 2 channel version we've fixed the 16 channel version, then we can do some *really* exciting stuff ;-)

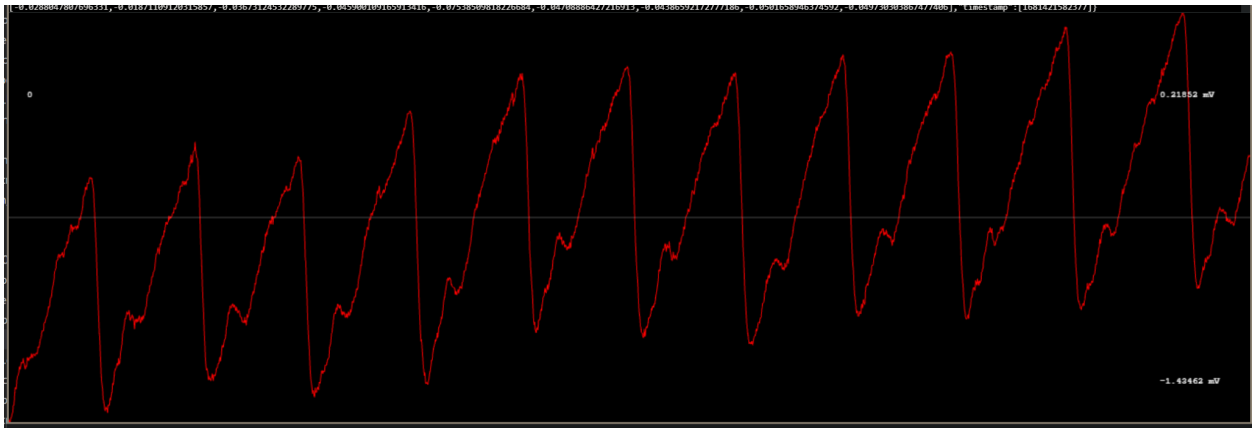


Note, another version features dedicated transimpedance amplifiers for each channel to help clean up the signal, but even without them (which is what we're testing right now) the data is state of the art.



16 channels is capable of a type of computed tomography (this is a web demo I made), which was developed as a low cost fMRI alternative. And someday this?

NIRS output (just a steady-on LED), these triangle waves are heart rhythm:

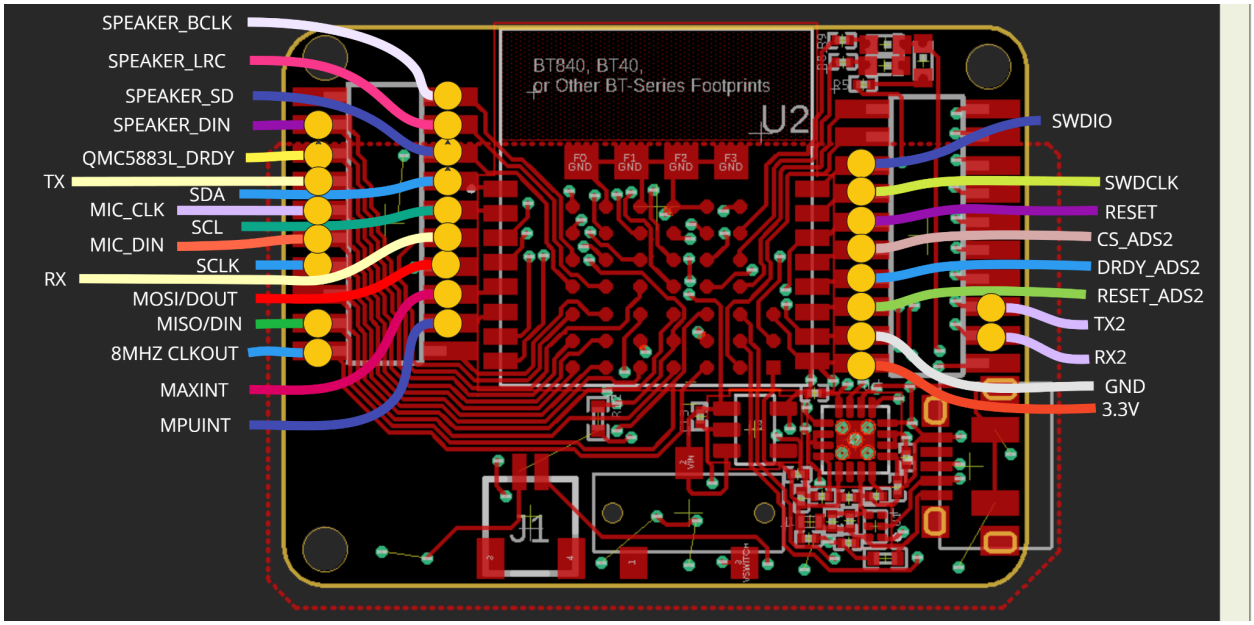


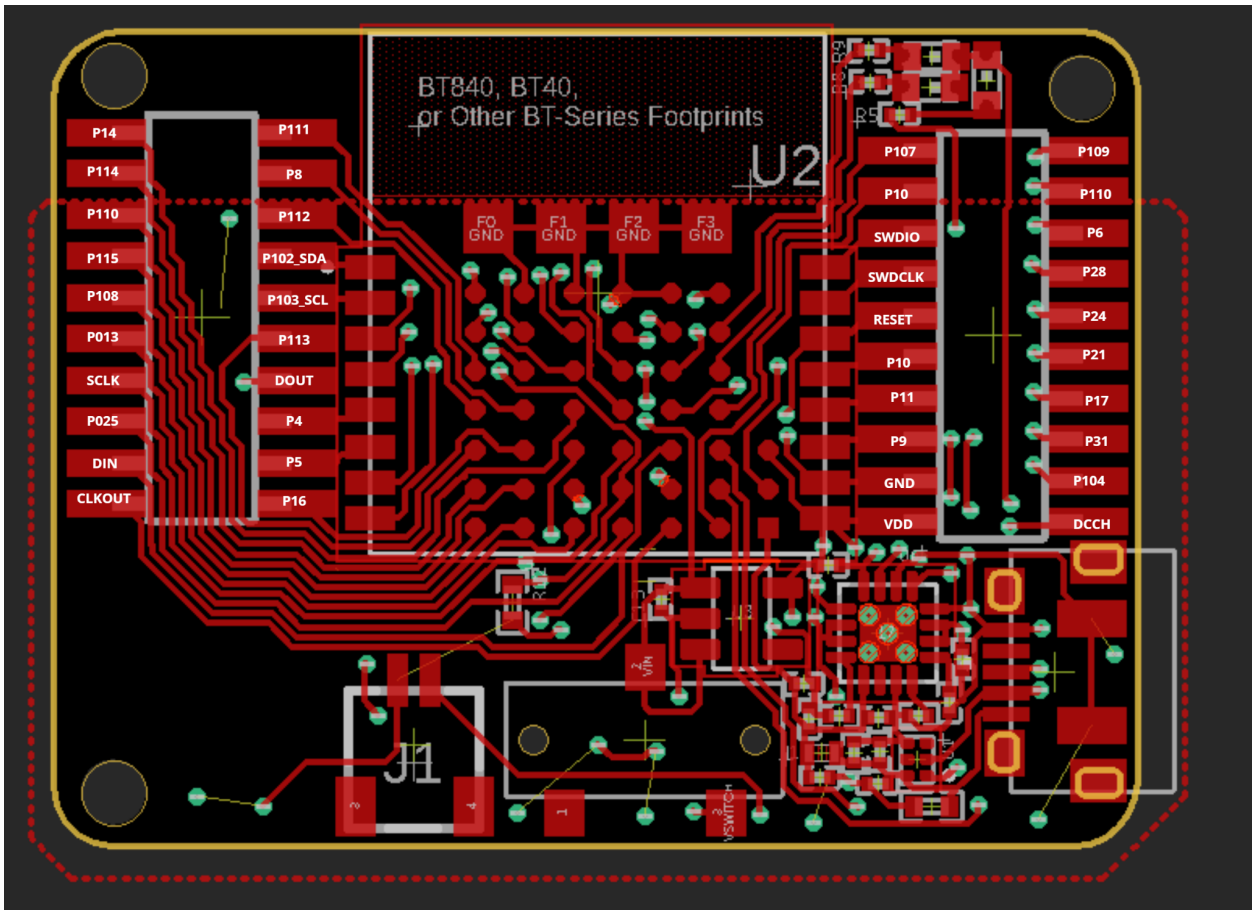
This is pristine PPG, you won't even see it this good in textbooks. This is an unfiltered 24 bit signal at 32X gain on the ADS131.

If we nail this, we'll get both signals simultaneously (not *quite* as high res on the PPG of course)

Each channel is labeled by which LED is reported over the BLE characteristic as a tag on the end of the simultaneous-sampling ADC readings. Then you subscribe to the first service and set the decoder to ADS131M08 to see the raw data stream in. There's a bug on channel 7 where the chart has to roll over once (IDK why).

Microcontroller: nRF5340. The pin beneath MISO/DIN is an 8.192MHz pin from the ADS131 crystal for driving additional ADCs or other operations.





BT40 pinouts

Chipset: https://www.fanstel.com/bt40f-nrf5340

Backside are the 8 channel ADC pins. They are labeled which end is 0 and which end is 7 on the board. The firmware supports all additional sensors plug-and-play with extensive BLE characteristic usage that we're still adding onto. They're configured to 32X gain right now, as that provides the best SNR with a more sensitive PGA at the higher gain configurations (Above 8X gain IIRC, best SNR at 32x-128x).

LMK what to clarify on, this is a huge rabbit hole but I'm just solving some basic problems around the hardware implementation while the overall designs are fairly clear cut at this stage.

The final product will integrate this all together into a couple PCBs, but for now we're keeping it simple with pluggable dev boards for just getting the FNIRs and EEG sampling in place.

Here is how you'll wire the nRFDK, this is the same for the nRF53 but SWO is not needed.

